Hindi Document RAG: A. P. J. Abdul Kalam — *Agni Ki Udaan*

Project Overview

This project implements a Retrieval-Augmented Generation (RAG) system that answers questions about the Hindi document *Agni Ki Udaan* using information retrieved from the document itself [cite: 1].

The system accepts questions in Hindi or English, retrieves relevant passages from the Hindi document, sends those passages to Gemini, and returns an answer with source metadata such as the PDF page number, section title, chunk ID, and retrieval similarity score [cite: 1].

The main challenge was not simply connecting an LLM to a vector database. The original document contains Hindi text, technical terms, tables, and PDF layout artifacts [cite: 1]. Incorrectly extracted text can lead to poor retrieval, so the project includes a document-preparation workflow before the RAG pipeline [cite: 1].

Main technologies

Component	**Technology**	**Purpose**
PDF extraction	PyMuPDF	Extract text and inspect PDF tables [cite: 1]
Text cleaning	Python, Unicode normalization, Rule A and Rule B	Prepare readable Hindi text [cite: 1]
Document storage	JSON	Preserve processed text, metadata, and chunks [cite: 1]
Embeddings	`BAAI/bge-m3`	Represent Hindi text and English questions as vectors [cite: 1]
Vector database	ChromaDB	Store vectors, text, and source metadata [cite: 1]

Component	Technology	Purpose
Answer generation	Google Gemini API	Generate answers from retrieved evidence [cite: 1]
Evaluation	Custom Python evaluation in `rag.py`	Measure retrieval performance on six benchmark questions [cite: 1]

---

1. Understanding the Complete Pipeline

The project has two major parts: document preparation and question answering [cite: 1].

```
None
ORIGINAL HINDI PDF
        |
        v
1. Extract PDF text and inspect tables
        |
        v
2. Rule A: clean selected duplicate vowel signs
        |
        v
3. Rule B: classify and reconstruct line boundaries
        |
        v
4. Extract and clean tables separately
        |
        v
5. Combine cleaned prose and tables
        |
        v
6. Create semantic chunks with metadata
        |
        v
7. Generate BGE-M3 embeddings
        |
        v
8. Store vectors and metadata in ChromaDB
        |
        v
       INDEX READY
        |
        v
```

```
USER ASKS A QUESTION IN HINDI OR ENGLISH
        |
        v
9. Embed the question and retrieve relevant chunks
        |
        v
10. Send retrieved evidence to Gemini
        |
        v
11. Generate an evidence-grounded answer
        |
        v
12. Attach citations using retrieved metadata
```

Each stage has a different responsibility [cite: 1]. Cleaning improves the text, chunking organizes it, embeddings represent its meaning numerically, ChromaDB retrieves relevant passages, and Gemini generates the final answer [cite: 1].

---

2. PDF Extraction and Text Preparation

2.1 Why PDF extraction needed attention

A PDF is primarily a document-layout format [cite: 1]. Extracting its text does not always reproduce the original reading order or word boundaries perfectly [cite: 1].

This matters especially for Hindi because Devanagari uses dependent vowel signs, conjuncts, and other Unicode characters [cite: 1]. An extraction error can cause a word to become incomplete or unreadable [cite: 1].

The workflow used separate raw, processed, and audit files rather than treating the first extraction as final [cite: 1].

2.2 Unicode normalization

Unicode allows some visually equivalent text to be represented in different ways [cite: 1]. NFC normalization converts eligible equivalent sequences into a consistent representation [cite: 1].

Python example:

```python
import unicodedata

text = unicodedata.normalize("NFC", text)
```

2.3 Rule A: Duplicate vowel-sign cleaning

Purpose: Rule A targets adjacent duplicate copies of selected Devanagari dependent vowel signs [cite: 1].

```
Input:  consonant + vowel sign + same vowel sign
                    |
                    v
Rule A: keep one copy of the adjacent duplicate
                    |
                    v
Output: consonant + vowel sign
```

2.4 Rule B: Generic line-boundary reconstruction

Purpose: Rule B handles line boundaries that may split words or phrases during PDF extraction [cite: 1].

Rule B classifies line boundaries into four decision types [cite: 1]:

- `KEEP_NEWLINE`: preserve the line break [cite: 1].
- `JOIN_WITH_SPACE`: join the lines with a space [cite: 1].
- `JOIN_NO_SPACE`: join without a space for limited cases [cite: 1].
- `REVIEW`: flag the boundary for manual inspection [cite: 1].

2.5 Table extraction and cleaning

The project handles tables separately from the prose surrounding them [cite: 1].
Table-processing notes record six cleaned tables with 55 rows in total, including headers [cite: 1], covering pages 9, 12, 16, 17, 20, and 22 [cite: 1].

---

3. Semantic Chunking

3.1 Why semantic chunking was chosen

Semantic chunking places boundaries where the content changes meaning or topic, rather than splitting blindly at fixed character lengths [cite: 1].

Configuration settings [cite: 1]:

- **Minimum chunk threshold**: About 250 characters [cite: 1]
- **Maximum chunk target**: About 1,100 characters [cite: 1]
- **Heading threshold**: About 120 characters [cite: 1]
- **Semantic split percentile**: 25th percentile [cite: 1]
- **Retrieval `TOP_K`**: 5 [cite: 1]

3.2 Why table rows are separate chunks

Keeping rows separate helps the retriever locate the relevant date or fact and preserves a clear citation target for timeline tables and awards [cite: 1].

---

4. Embeddings and ChromaDB

- **Model**: `BAAI/bge-m3` multilingual embedding model [cite: 1].
- **Database**: ChromaDB local persistent storage [cite: 1].
- **Collection**: `kalam_hindi_semantic` [cite: 1].
- **Storage Directory**: `data/processed/chroma_db_semantic` [cite: 1].

---

5. Retrieval, Answer Generation & Evaluation

- **Retrieval Evaluation Metrics** on six benchmark questions [cite: 1]:
  - **Hit@1**: 5/6 — 83.3% [cite: 1]
  - **Hit@3**: 6/6 — 100% [cite: 1]
  - **Hit@5**: 6/6 — 100% [cite: 1]
  - **MRR**: 0.917 [cite: 1]

---

6. Installation and Usage

Step 1: Virtual environment setup

```
None
python -m venv RAG
.\RAG\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Step 2: Environment configuration (`.env`)

```
None
GEMINI_API_KEY=your_api_key_here
GEMINI_MODEL=gemini-2.5-flash
```

Step 3: Pipeline Commands

```
None
python rag.py chunk
python rag.py index
python rag.py answer "कलाम को भारत रत्न किस वर्ष प्राप्त हुआ?"
python rag.py answers
python rag.py eval
```

---

7. Web Application (FastAPI)

Run the local web UI backend [cite: 1]:

```
None
python -m uvicorn api:app --reload
```

Open http://127.0.0.1:8000 in your browser [cite: 1].